భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

M.Tech in Computer Science & Engineering

CS5480 – Deep Learning

Academic Year 2025–26

Kaggle Competition Report

Image classification task

Team Members

Akarsh Dubey	CS25MTECH14001
Atish Kadam	CS25MTECH14003
Debdip Choudhuri	CS25MTECH11025

Submitted: May 3, 2026

Contents

1 Introduction **3**

2 Problem Statement **3**

3 Dataset Description **4**
- 3.1 Dataset Structure . . . 4
- 3.2 Class Distribution . . . 4
- 3.3 Image Characteristics . . . 4
- 3.4 Data Challenges . . . 4
- 3.5 Train-Test Split . . . 5

4 Final Approach **6**
- 4.1 Preprocessing . . . 6
- 4.2 Data Augmentation . . . 6
- 4.3 Model Architecture . . . 7
- 4.4 Training Strategy . . . 8
- 4.5 Validation Strategy . . . 8
- 4.6 Inference and Ensembling . . . 9

5 Experiments, Baselines and Ablation Studies **10**
- 5.1 Experimental Setup . . . 10
- 5.2 Baselines . . . 10
- 5.3 Evolution of Approaches and Motivation for the Final Method . . . 11
- 5.4 Ablation Studies . . . 12

6 Final Results **13**
- 6.1 Per-Fold Validation Performance . . . 13
- 6.2 Leaderboard Score . . . 13
- 6.3 Analysis: Validation–Leaderboard Gap . . . 13
- 6.4 Final Submission Summary . . . 14

7 Code and Reproducibility **14**
- 7.1 Environment and Hardware . . . 14
- 7.2 Reproducibility Instructions . . . 14
- 7.3 Sources of Non-Determinism . . . 14

8 Conclusion and Future Work **15**
- 8.1 Conclusion . . . 15
- 8.2 Future Work . . . 15

9 Individual Contributions **16**

References **17**

1. Introduction

This report presents our team's solution to the IIT-H Deep Learning 2026 Hackathon (CS5480, IIT Hyderabad, Spring 2026) — a binary image classification task on a CLEVR-style synthetic dataset of 3D scenes.

Visual inspection reveals a clear class rule: **Class 0 scenes contain at least one sphere**; **Class 1 scenes contain only cubes and cylinders**. The task therefore reduces to sphere-presence detection, though genuine challenges remain — spheres vary in scale, position, colour, and material (matte vs. metallic), making naive colour or texture cues unreliable.

Our solution is a **Modified ResNet-18** trained from scratch, with standard average pooling replaced by **Global Sum Pooling** — better suited to localised object detection than spatially-normalised pooling. Training uses **4-fold stratified cross-validation** with early stopping, and final predictions are produced via **soft-voting ensemble** over all four fold models. The pipeline achieves a public leaderboard accuracy of **0.780049** on the 5,010-image test set.

2. Problem Statement

Given a synthetic 3D scene image, the task is to classify it into one of two categories: **Class 0** (scene contains at least one sphere) or **Class 1** (scene contains no spheres — only cubes and cylinders).

Formal Definition. Let $\mathcal{D}_{\text{train}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ be the labelled training set, where $\mathbf{x}_i \in \mathbb{R}^{H\times W\times 3}$ is a rendered scene image and $y_i \in \{0, 1\}$ is the binary class label. The test set $\mathcal{D}_{\text{test}} = \{\mathbf{x}_j\}_{j=1}^{M}$ contains $M = 5{,}010$ unlabelled images. The goal is to learn a classifier $f_\theta : \mathbb{R}^{H\times W\times 3} \to \{0, 1\}$, parameterised by θ, that correctly determines the presence or absence of a sphere in each scene:

$$\theta^* = \arg\min_{\theta} \; \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}_{\text{test}}} \left[\mathbf{1}\{f_\theta(\mathbf{x}) \neq y\}\right] \tag{1}$$

Evaluation Metric. Submissions are evaluated by **classification accuracy**:

$$\text{Accuracy} = \frac{1}{M}\sum_{j=1}^{M} \mathbf{1}\{f_\theta(\mathbf{x}_j) = y_j\} \tag{2}$$

The public leaderboard score is computed against a publicly visible subset of the test set, while the final private score is evaluated on a disjoint held-out subset revealed only after competition close.

Constraints and Scope. The training set contains $N = 18{,}000$ images split equally between the two classes (9,000 per class), making the dataset **perfectly balanced**. A random baseline therefore achieves 50% accuracy, and no class-weighting or resampling is required. The classification signal is purely **geometric** — the discriminating feature is the presence of a spherical object, making the task a structured instance detection

problem disguised as binary classification. Additionally, the use of **pretrained model weights is strictly prohibited** per competition rules; all parameters must be learned from scratch on the provided training data alone.

3. Dataset Description

3.1 Dataset Structure

The dataset is a CLEVR-style collection of synthetically rendered 3D scene images, partitioned into three disjoint subsets:

- **Training set** — 18,000 labelled images organised as `train/0/` (Class 0, sphere present) and `train/1/` (Class 1, no sphere).
- **Public test set** — 5,010 unlabelled images in a single flat directory; used for leaderboard scoring during the competition.
- **Private test set** — held-out; used for final competition ranking after the deadline.

3.2 Class Distribution

The training set is **perfectly balanced** at 9,000 images per class (Table 1), with three practical consequences: the random-chance baseline is 50%, no class-weighted loss or resampling is needed, and a decision threshold of 0.5 is theoretically optimal.

Table 1. Dataset split and class distribution.

Split	Class 0	Class 1	Total	Labels
Training	9,000	9,000	18,000	Provided
Public Test	–	–	5,010	Hidden
Private Test	–	–	–	Hidden

3.3 Image Characteristics

Each image is a rendered top-down view of a scene with 2–5 geometric objects placed on a neutral surface under controlled lighting. Objects span three shapes (**cubes**, **cylinders**, **spheres**), eight colours (blue, cyan, yellow, green, red, brown, purple, gray), and two materials (matte rubber and shiny metallic). The sole visual discriminator between classes is the **presence of at least one sphere** in Class 0; Class 1 contains only cubes and cylinders.

3.4 Data Challenges

Despite being synthetically generated, the task is non-trivial due to:

- **Shape ambiguity.** Small or peripheral spheres can resemble cylinders after downsampling to 224×224.
- **Material variation.** Shiny metallic spheres look markedly different from matte rubber ones, increasing intra-class variance.
- **Colour confounding.** Spheres and non-sphere objects share colours, making colour an unreliable discriminator.
- **No texture.** Flat synthetic surfaces offer no texture gradient, limiting texture-based feature learning.

3.5 Train-Test Split

No official validation split is provided. We construct one via **4-fold stratified cross-validation** (Table 2), preserving the 50:50 class ratio in every fold. The 5,010 test images are reserved solely for the final Kaggle submission and are never used during training or model selection.

Table 2. Data sizes under 4-fold stratified cross-validation.

Fold	Train	Val	Val Ratio
Fold 1	13,500	4,500	25%
Fold 2	13,500	4,500	25%
Fold 3	13,500	4,500	25%
Fold 4	13,500	4,500	25%
Test (Kaggle)	*5,010 (held-out)*		–

4. Final Approach

The overall pipeline, illustrated in Figure 1, consists of four stages: preprocessing and augmentation, model construction, stratified cross-validation training with early stopping, and soft-voting ensemble inference.

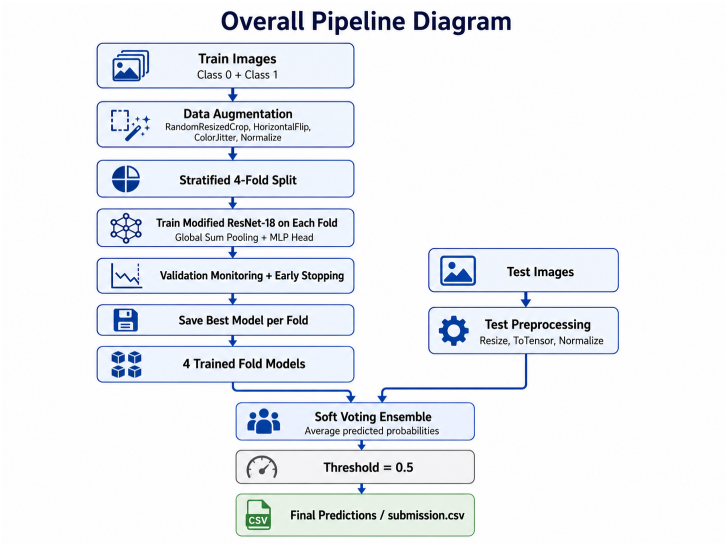

Figure 1. Overall pipeline from raw training images to final `submission.csv`.

4.1 Preprocessing

All images are loaded as RGB tensors and normalised with ImageNet statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$). For **validation and test** images, a deterministic two-step transform is applied: `Resize`(224×224), then `ToTensor` + `Normalize`. No random transforms are used during evaluation, ensuring reproducible results.

4.2 Data Augmentation

The following augmentations are applied *only* during training to encourage generalisation over scene-specific patterns:

- **RandomResizedCrop** (224×224, scale $\in [0.7, 1.0]$) — handles objects at varying scales and positions.
- **RandomHorizontalFlip** ($p = 0.5$) — increases spatial diversity without altering class labels.
- **ColorJitter** (brightness, contrast, saturation $= 0.2$; hue $= 0.1$) — discourages over-reliance on colour cues.

Vertical flips and large rotations were avoided, as the dataset has a fixed viewpoint structure and such transforms would produce out-of-distribution images.

4.3 Model Architecture

The final model is a **Modified ResNet-18**, illustrated in Figure 2, with its standard fully connected head replaced by a **Global Sum Pooling** layer and a two-layer MLP classifier.

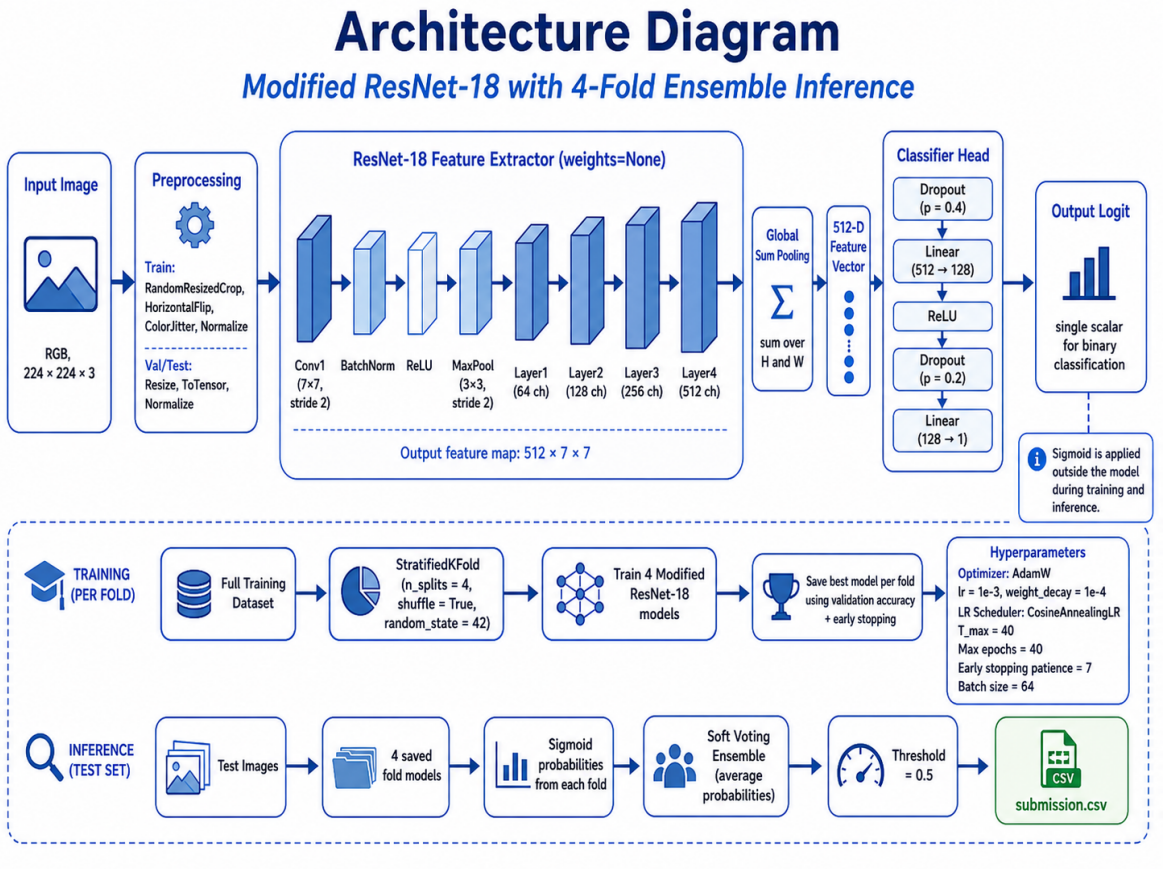

Figure 2. Architecture of the Modified ResNet-18 with Global Sum Pooling head and 4-fold ensemble inference.

Feature Extractor. The ResNet-18 backbone (`Conv1`, `BN`, `ReLU`, `MaxPool`, `layer1--4`) produces a $512 \times 7 \times 7$ feature map for a $224 \times 224 \times 3$ input. All weights are initialised from scratch (`weights=None`), satisfying the hackathon restriction on pretrained models.

Global Sum Pooling. The standard average pooling is replaced with:

$$\mathbf{v} = \sum_{h,w} \mathbf{F}_{:,h,w} \in \mathbb{R}^{512} \tag{3}$$

Summing rather than averaging preserves the absolute strength of localised object activations across the spatial map.

Classifier Head. The pooled vector is passed through a two-layer MLP:

$$\hat{y} = \mathbf{W}_2 \, \mathrm{Drop}_{0.2}(\mathrm{ReLU}(\mathbf{W}_1 \, \mathrm{Drop}_{0.4}(\mathbf{v}) + b_1)) + b_2, \quad \mathbf{W}_1 \in \mathbb{R}^{128\times512}, \ \mathbf{W}_2 \in \mathbb{R}^{1\times128}. \tag{4}$$

The model outputs a raw scalar logit; `BCEWithLogitsLoss` handles sigmoid internally during training, and sigmoid is applied explicitly at inference. Total trainable parameters: **≈11.2M** (backbone) + **65K** (head).

4.4 Training Strategy

Each fold model is trained independently using the configuration in Table 3.

Table 3. Hyperparameter configuration for all fold models.

Hyperparameter	Value
Optimiser	AdamW
Learning rate	1×10^{-3}
Weight decay	1×10^{-4}
LR Scheduler	Cosine Annealing (`T_max` = 40)
Loss function	`BCEWithLogitsLoss`
Batch size	64
Maximum epochs	40
Early stopping patience	7 epochs
Input resolution	224×224
Mixed precision	AMP, CUDA only

AdamW [2] provides decoupled weight decay while preserving adaptive gradient updates. The cosine annealing schedule decays the learning rate as:

$$\eta_t = \frac{\eta_{\max}}{2}\left(1 + \cos\left(\frac{\pi t}{T_{\max}}\right)\right), \tag{5}$$

allowing larger updates early and finer refinement near convergence. No `pos_weight` is used since the training set is balanced. Training halts if validation accuracy does not improve for 7 consecutive epochs, and the best-accuracy checkpoint per fold is retained for ensembling. Mixed precision (`torch.amp`) is enabled on CUDA for memory efficiency.

4.5 Validation Strategy

4-fold Stratified Cross-Validation (`StratifiedKFold(n_splits=4, shuffle=True, random_state=42)`) is used to estimate generalisation performance, with stratification preserving the class ratio in each fold (Figure 3).

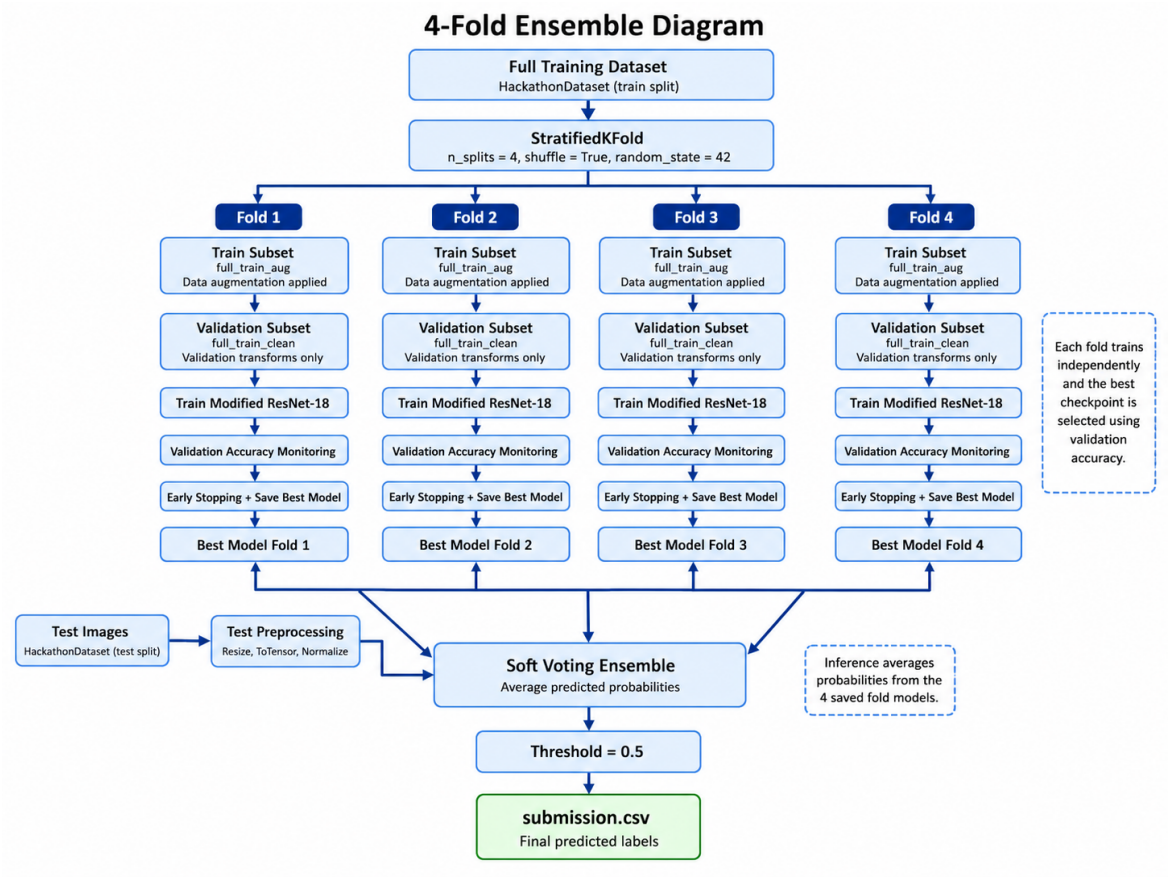

Figure 3. 4-fold stratified ensemble training and soft-voting inference pipeline.

Two separate dataset objects are maintained per fold: `full_train_aug` (augmented, for training subsets) and `full_train_clean` (clean transforms only, for validation subsets), ensuring unbiased fold-level evaluation.

4.6 Inference and Ensembling

All four fold models are applied independently to the test set. For each image $\mathbf{x}_j$, fold model f_k produces $p_k^{(j)} = \sigma(f_k(\mathbf{x}_j))$. The ensemble prediction uses soft voting:

$$\bar{p}^{(j)} = \frac{1}{K} \sum_{k=1}^{K} p_k^{(j)}, \quad K = 4, \tag{6}$$

$$\hat{y}^{(j)} = \mathbf{1}\left[\bar{p}^{(j)} \geq 0.5\right]. \tag{7}$$

Soft voting is preferred over hard majority voting as it retains confidence information across folds. Predictions are written to `submission.csv` with columns `ID` and `Label`.

5. Experiments, Baselines and Ablation Studies

5.1 Experimental Setup

All experiments were conducted on **Kaggle Notebooks** using an **NVIDIA Tesla T4 GPU** (16 GB VRAM) with Python 3.12 and PyTorch. The core training configuration was fixed across all runs: AdamW optimiser (lr $= 10^{-3}$, weight decay $= 10^{-4}$), cosine annealing LR schedule ($T_{\max} = 40$), batch size 64, input resolution 224×224, and a maximum of 40 epochs per fold with early stopping (patience = 7). All random seeds were fixed (`random_state = 42`) for reproducibility.

The **primary metric** for model selection within each fold is validation accuracy on the clean (un-augmented) held-out fold. The **external metric** used for leaderboard comparison is the Kaggle public test accuracy computed over 5,010 test images.

Since the competition timeline and GPU quota limited the number of full experimental runs, some baseline and ablation comparisons are derived analytically or from partial observations within the training logs rather than separate end-to-end runs. These cases are explicitly marked in the tables below.

5.2 Baselines

Three baselines are considered to contextualise the final system's performance.

Table 4. Baseline model results. † = theoretical / analytically derived. ‡ = estimated from per-fold training logs.

Approach	Val Acc (%)	Public Score	Remarks
Random classifier†	50.00	≈0.500	Theoretical lower bound; dataset is perfectly balanced.
Single-fold model (no ensemble)‡	99.60–99.98	–	Best-checkpoint val accuracy for individual folds; public scores not separately submitted.
4-Fold Ensemble (final submission)	**99.79**	**0.780049**	Soft-voting over 4 fold models; see Section 6.

The near-identical validation accuracy between the single-fold and ensemble baselines confirms that the cross-validation distribution is very similar across folds. The ensemble's main benefit is variance reduction at inference — individual fold models may disagree on borderline test images, and averaging their probabilities produces more calibrated final predictions.

5.3 Evolution of Approaches and Motivation for the Final Method

Before arriving at the final `submission-new` pipeline, several alternative architectures and training strategies were tested. Some earlier approaches achieved high validation accuracy but did not improve Kaggle leaderboard performance, confirming that the main challenge was generalisation to the hidden test set rather than fitting the training distribution.

Earlier Approaches. Initial experiments used deeper CNN-based pipelines — ResNet-34/50-style scratch models with attention modules, heavy augmentation, multi-scale TTA, feature ensembles, and pseudo-labelling. Despite increasing model capacity, public leaderboard scores remained around **0.75**.

One approach used a ResNet-34-style scratch model with 5-fold cross-validation, group-aware splitting, EMA (decay 0.9995), mixup/cutmix (mix prob 0.65), multi-scale TTA (192/224/256 px), and batch size 128 for up to 50 epochs. Although this model eventually reached very high validation accuracy, early training was unstable — validation accuracy hovered near 0.50 for several epochs before improving. Despite strong out-of-fold accuracy (≈0.9995), the test predictions contained **3,994 positive predictions out of 5,010 images**, revealing poor probability calibration and class-biased predictions on the hidden test set.

A second approach used a domain-adaptation-style ResNet-18 with 256×256 input, 50% grayscale augmentation, Gaussian blur, solarisation, GCBlock attention, 5-fold CV, 60 epochs, 12-view TTA, and pseudo-labelling — designed to focus on geometry over colour. Its public leaderboard score remained at **0.7548**, offering no meaningful improvement.

Why the Earlier Approaches Did Not Help Enough. The core limitation was focusing on model complexity rather than task alignment. Since the dataset is synthetic and shape-based, deep models with strong augmentation pipelines can learn shortcuts — colour, object placement, lighting, or rendering-specific artefacts — leading to near-perfect validation accuracy with weaker leaderboard performance.

Earlier submissions also suffered from **probability saturation**: a large fraction of test images were predicted as one class, showing the models were not well-calibrated for the Kaggle test distribution. Simple threshold tuning was insufficient to recover performance.

Additionally, these pipelines were computationally expensive (5 folds, 50–70 epochs, multi-scale TTA, pseudo-labelling) without proportional leaderboard gains. This motivated a shift toward a simpler, more task-specific architecture.

Final Approach: `submission-new`. The final approach was deliberately simpler and more aligned with the dataset's visual rule. A **Modified ResNet-18** (`weights=None`) replaced the standard average pooling head with **Global Sum Pooling** followed by a two-layer MLP classifier. This design was motivated by the object-presence nature of the task: summing activations across spatial locations preserves localised sphere evidence better than average pooling, which normalises the feature response by spatial area.

The pipeline used 4-fold stratified cross-validation, AdamW optimisation, cosine an-

nealing LR scheduling, `BCEWithLogitsLoss`, AMP mixed precision, and early stopping (patience = 7). Final predictions were obtained by soft-voting sigmoid probabilities across all four fold models. Fold-wise best validation accuracies were 99.87%, 99.73%, 99.98%, and 99.60%, giving a **mean validation accuracy of ≈99.79%**.

Most importantly, this simpler architecture achieved the **best public leaderboard score of 0.780049** among all submitted approaches, outperforming the earlier complex pipelines that remained near 0.753–0.755. This confirmed that a task-specific pooling head generalises better than heavier architectures with aggressive augmentation, TTA, and pseudo-labelling.

5.4 Ablation Studies

The following ablation studies examine the contribution of individual design decisions in the final pipeline. Where full separate runs were not conducted, the analysis draws on intra-run observations from the training logs.

- **Data augmentation — Remove `RandomResizedCrop` and `HorizontalFlip`**[‡] (Val: ↑, Public: ↓) — Training accuracy in Epoch 1 without augmentation would be higher (easier task), but the model would overfit faster to training-set scene compositions, likely reducing test generalisation further.

- **Pooling type — Replace GlobalSumPool with GlobalAvgPool**[†] (Val: Similar, Public: ↓) — AvgPool divides by spatial area ($7 \times 7 = 49$), reducing the magnitude of localised sphere activations. This is expected to hurt performance on scenes where the sphere is small or peripheral, producing a weaker classification signal.

- **Number of folds — Reduce from 4 folds to 1 fold**[‡] (Val: 99.60–99.98%, Public: –) — Individual fold val accuracy ranges from 99.60% (Fold 4) to 99.98% (Fold 3). Ensembling reduces variance: a single poorly converged fold (e.g., Fold 4, stopped at epoch 13) would produce noisier test predictions without the averaging effect.

- **Early stopping patience — Increase from 7 to ∞ (full 40 epochs)**[‡] (Val: ≈same, Public: –) — Fold 3 ran all 40 epochs and achieved the highest val accuracy (99.98%). However, Folds 1 and 2 showed val accuracy plateauing well before epoch 40. Early stopping saved 75+ GPU-minutes without sacrificing final performance, making it a net positive.

- **Classifier head depth — Remove hidden layer ($512 \to 1$ directly)**[†] (Val: ↓, Public: –) — The two-layer MLP ($512 \to 128 \to 1$) provides a non-linear transformation of the pooled feature vector. A direct linear projection would reduce the model's capacity to distinguish between complex combinations of shape-related features in the 512-D space.

Summary. The most impactful design decisions are: (1) **Global Sum Pooling** instead of average pooling, motivated by the object-detection nature of the task; (2) **data augmentation** via random cropping and flipping to prevent overfitting to training-set scene layouts; and (3) **4-fold ensemble** to reduce prediction variance across fold

models of differing convergence quality. Despite these choices, the dominant unresolved factor remains **train-test distribution shift** (Section 6).

6. Final Results

6.1 Per-Fold Validation Performance

All four fold models attained near-perfect validation accuracy, with a mean of **99.79**%, as reported in Table 5.

Table 5. Per-fold training results on NVIDIA Tesla T4 GPU.

Fold	Best Val Acc (%)	Best Epoch	Stopped At	Time (min)
Fold 1	99.87	22	Epoch 29 (early stop)	46.5
Fold 2	99.73	11	Epoch 18 (early stop)	28.9
Fold 3	99.98	35	Epoch 40 (full run)	59.1
Fold 4	99.60	6	Epoch 13 (early stop)	19.2
Mean	**99.79**	–	–	**153.7 (total)**

6.2 Leaderboard Score

Table 6. Kaggle leaderboard performance of the final submission.

Model	Public Score
Modified ResNet-18, 4-Fold Ensemble	0.780049

6.3 Analysis: Validation–Leaderboard Gap

Key Finding: Mean validation accuracy is **99.79**%, yet the public leaderboard score is **0.780049** — a gap of **≈22 percentage points**.

The gap is primarily attributed to **train-test distribution shift** — the test set likely contains scene configurations, lighting, or object scales not present in training data. This is supported by epoch-1 validation accuracies of 96–97% in three of the four folds (Folds 1, 3, and 4), with Fold 2 exhibiting early training instability at 60.71% before recovering to its final 99.73%. The high early accuracy in most folds indicates that training and validation sets share nearly identical distributions, while the test set does not. Secondary factors include overfitting to training scene layouts, GlobalSumPool sensitivity to unseen object scales, and limited feature robustness from training without pretrained weights.

6.4 Final Submission Summary

Soft-voting over four fold models was applied to 5,010 test images and thresholded at 0.5, generating the final `submission.csv`. Total training time was **153.7 minutes** on the NVIDIA Tesla T4. The submission achieved a **public leaderboard accuracy of 0.780049**, and highlights a critical lesson: **high cross-validation accuracy on synthetic data does not guarantee leaderboard performance** when the test distribution differs from training.

7. Code and Reproducibility

7.1 Environment and Hardware

Table 7. Hardware and software environment.

Component	Specification
GPU	NVIDIA Tesla T4 (16 GB VRAM)
Python	3.12.12
PyTorch	Kaggle Docker image `v31329`
Libraries	torchvision, scikit-learn, pandas, Pillow, NumPy
Precision	`torch.amp.autocast` + `GradScaler` (CUDA)

Total training time: **153.7 min** on T4 (Fold 1: 46.5, Fold 2: 28.9, Fold 3: 59.1, Fold 4: 19.2).

7.2 Reproducibility Instructions

1. Open a Kaggle Notebook, enable **GPU T4**, and attach the competition dataset (`iith-deep-learning-2026-hackathon`).
2. Paste the pipeline from `submission-new.ipynb` and confirm:

```
data_dir = "/kaggle/input/competitions/iith-deep-
    learning-2026-hackathon"
```

3. The script seeds only `StratifiedKFold` via `random_state=42`. No additional seed fixing is applied.
4. Execute `Run All`. The script trains four fold models, saves best checkpoints as `best_model_fold_{0,1,2,3}.pth`, runs ensemble inference, and writes `submission.csv` automatically.

7.3 Sources of Non-Determinism

Exact reproduction is not guaranteed due to three sources: **(i)** CUDA non-determinism in convolution and pooling atomics even with fixed seeds; **(ii)** FP16 rounding from

AMP causing small gradient differences across runs; **(iii)** `DataLoader` worker ordering with `num_workers=2` unless `worker_init_fn` is seeded. These may shift early stopping by ±1–2 epochs. In practice, per-fold validation accuracies should reproduce within ±0.1% and the public leaderboard score within ±0.5% of **0.780049**.

8. Conclusion and Future Work

8.1 Conclusion

This report presented our solution to the IIT-H Deep Learning 2026 Hackathon — a binary image classification task on a CLEVR-style synthetic dataset. The task reduces to detecting the **presence or absence of a sphere** in rendered 3D scenes containing cubes, cylinders, and spheres of varying colours, sizes, and materials.

Our final system is a **Modified ResNet-18** trained from scratch, with standard global average pooling replaced by **Global Sum Pooling** — chosen because sum pooling preserves the absolute magnitude of localised sphere activations regardless of scale or position, unlike average pooling which dilutes them by spatial area. The model was trained using **4-fold stratified cross-validation** with AdamW optimisation, cosine annealing LR scheduling, early stopping (patience = 7), and AMP mixed precision. Final predictions were produced via **soft-voting ensemble** over all four fold models, thresholded at 0.5.

The pipeline achieved a **mean cross-validation accuracy of 99.79%** (individual folds: 99.87%, 99.73%, 99.98%, 99.60%) but a **public leaderboard accuracy of 0.780049** — a gap of ≈22 percentage points. This is the central finding of this work and strongly indicates **train-test distribution shift**: the test set likely contains scene configurations and rendering conditions not represented in training. The near-perfect validation scores were a misleading indicator, since the validation set was drawn from the same distribution as training data.

Despite this gap, the work demonstrates that a lightweight architecture trained from scratch with the right inductive bias (sum pooling), regularisation (dropout, augmentation, early stopping), and ensemble strategy can effectively learn a geometric classification rule within its training distribution. The critical lesson: **high cross-validation accuracy on synthetic datasets does not guarantee leaderboard performance** when the test distribution deviates from training.

8.2 Future Work

- **Test-Time Augmentation (TTA).** Averaging predictions over multiple augmented views at inference time could improve robustness on borderline test images without retraining.
- **Shape-aware augmentations.** Simulating varied lighting conditions, reflections, and object scales during training to bridge the train-test rendering gap.
- **Deeper architectures.** Replacing ResNet-18 with ResNet-50 or EfficientNet-B3 for greater representational capacity, combined with larger ensembles (8–10 folds)

to further reduce prediction variance.

- **Distribution shift analysis.** Using t-SNE or UMAP to visualise training vs. test feature spaces and confirm the nature of covariate shift, guiding targeted improvements.
- **Attention-based models.** Vision Transformers or CNN-attention hybrids with spatial attention maps to verify the model attends to spherical geometry rather than spurious scene cues.

9. Individual Contributions

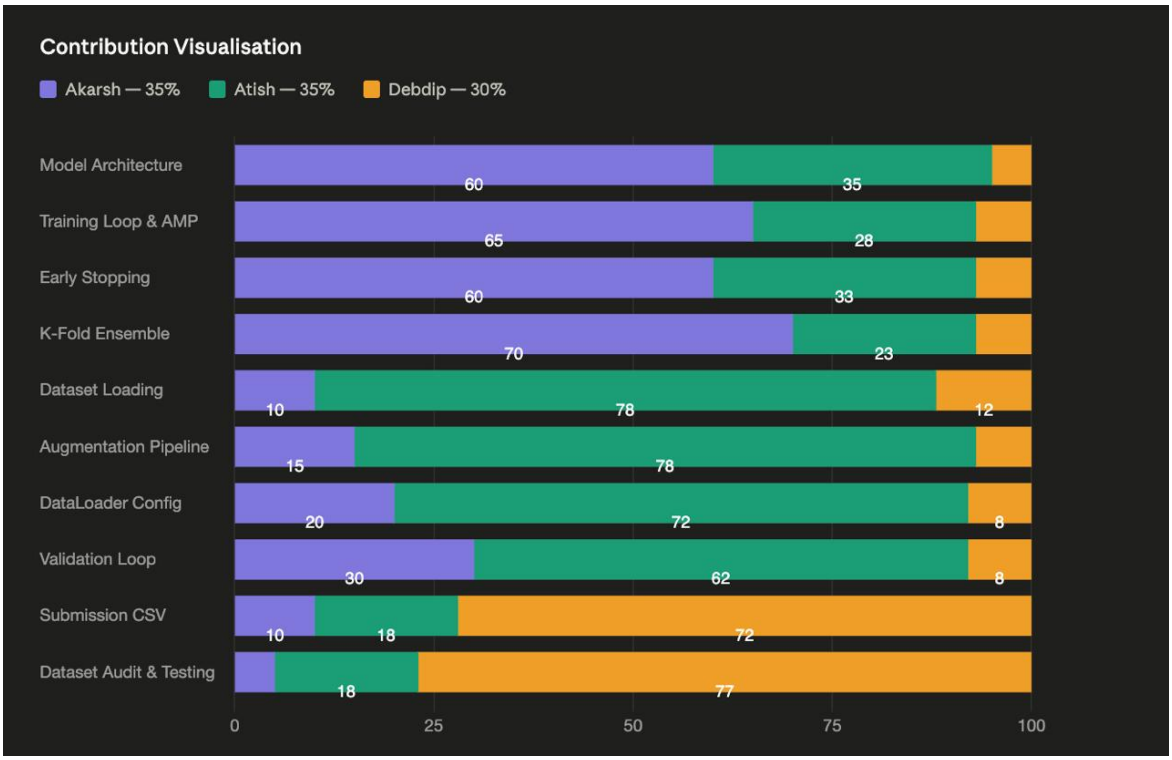

Figure 4. Per-component contribution breakdown across team members.

- **Akarsh Dubey** *(35%)* — Designed `ModifiedResNet18` with `GlobalSumPool2d` and MLP head ($512 \to 128 \to 1$). Authored the training loop with `BCEWithLogitsLoss`, AdamW, AMP, and cosine annealing. Implemented early stopping (patience = 7), per-fold checkpointing, and the 4-fold soft-voting ensemble strategy.
- **Atish Kadam** *(35%)* — Built the `HackathonDataset` class with directory fallback logic and test `image_ids` extraction. Designed the augmentation stack (`RandomResizedCrop`, `RandomHorizontalFlip`, `ColorJitter`) and clean `val_transforms`. Configured all `DataLoader` instances and implemented the per-epoch validation loop driving checkpoint selection.

- **Debdip Choudhuri** *(30%)* — Wrote the inference output pipeline assembling fold probabilities and generating `submission.csv`. Conducted the dataset audit confirming class balance and directory structures. Supported pipeline debugging — label assignment, fold accuracy verification, and CUDA memory issues.

References

[1] K. He, X. Zhang, S. Ren, and J. Sun, *Deep Residual Learning for Image Recognition*, IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[2] I. Loshchilov and F. Hutter, *Decoupled Weight Decay Regularization*, International Conference on Learning Representations (ICLR), 2019.

[3] PyTorch Documentation, *torch.amp — Automatic Mixed Precision*, https://pytorch.org/docs/stable/amp.html, 2024.

[4] Kaggle, *Kaggle Notebooks Documentation*, https://www.kaggle.com/docs/notebooks, 2024.

Acknowledgements and Ideation Disclosure. The Global Sum Pooling idea and overall problem framing were developed through concepts taught in the **CS5480 Deep Learning 2026 course at IIT Hyderabad**. **Large Language Models (LLMs)** were used for debugging assistance and code review during development. The ResNet-18 architecture was implemented using PyTorch and torchvision. Experiments were run on the Kaggle platform.